

实验5：进程管理与上下文切换

第一个用户态进程——proczero 的诞生

概述

项目	说明
实验编号	Lab 5 of 8
框架目录	whu-oslab-skeleton/
需要修改的框架文件	kernel/proc/proc.c kernel/proc/swtch.S kernel/trap/trap.c
需要自行创建的文件	user/proczero.c （或 user/initcode.S） 、以及任何你认为需要的新函数
验收标准	make run 后： ① 系统维持时钟中断正常（打印 Tick!）； ② 第一个 CPU 核心成功进入用户态进程 proczero，并在用户态中执行至少两次系统调用，系统捕获并打印对应的系统调用信息。

本实验的挑战性质

实验1-4 提供了完整的 TODO 框架，学生补全具体语句。

本实验不同：内核调度部分（任务A）继续提供 TODO 框架，而用户态进程部分（任务B~E）需要学生自主设计并创建新函数/新文件，框架中没有对应的 TODO 槽位。手册提供充分的理论原理和设计约束，但不给出代码模板。

第一节：为什么需要用户态进程？

在前四个实验中，所有代码都运行在 S-Mode（内核模式）下。内核可以随意读写任何内存、执行特权指令。这种设计在单任务演示系统中可以接受，但一旦要运行不可信的用户程序，就会面临根本性的安全问题：

如果用户程序与内核共享特权级，一个有漏洞的（或恶意的）用户程序可以覆盖内核数据、读取其他进程的内存，甚至关闭中断使系统僵死。

RISC-V 的 U-Mode（用户模式）通过硬件强制隔离解决了这个问题：

- U-Mode 下的代码不能执行特权指令（如 csrw）
- U-Mode 下的内存访问受页表权限控制（没有 U 位的页不可访问）

- U-Mode 想要调用内核服务**唯一的合法途径**是 `ecall` 指令（触发异常，强制返回 S-Mode）

这就是“系统调用”机制的硬件基础，也是本实验的核心。

第二节：理论基础——两种不同的状态保存机制

理解本实验最重要的前提是区分两种“保存状态”的机制，它们长得相似但目的完全不同：

2.1 `struct context` ——内核线程之间的主动切换

当内核主动让出 CPU（通过 `sched()` → `swtch()`），它需要保存：

- **被调用者保存寄存器（Callee-saved）**：`ra`，`sp`，`s0` - `s11`

为什么只保存这 14 个？因为 `swtch()` 是一个普通的 C 函数调用。按照 RISC-V ABI，调用者在调用前已经保存了临时寄存器（`t0` - `t6`，`a0` - `a7`）；被调用者（`swtch`）只需要保证 Callee-saved 寄存器不被破坏。

`swtch` 的“魔法”：它不是真的“保存然后恢复”，而是把 `sp` 偷换成另一个进程的栈，然后从对方的 `ra` 跳走——世界就在 `ret` 的那一瞬间切换了。

2.2 `struct trapframe` ——被陷阱强行打断时的完整现场

当用户程序触发陷阱（`ecall`、缺页、外部中断），CPU 硬件强行将控制权交给内核。此时需要保存所有 32 个通用寄存器，因为：

- 用户程序不知道自己会被打断
- 内核处理完后必须能让用户程序“若无其事地继续”
- 哪怕是一个临时寄存器 `t3` 的值都不能丢失

`trapframe` 还存储了额外的“过境信息”：

```
1 kernel_sp    ← 内核栈顶地址（陷入后切换到内核栈）
2 kernel_trap  ← usertrap() 函数的地址
3 kernel_satp  ← 内核页表（切换地址空间用）
4 epc          ← 陷入时的用户 PC（即 ecall 指令的地址）
```

设计问题（在动手前请书面回答）：

1.

当用户程序从 `ecall` 返回后，为什么 PC 不能仍然指向 `ecall` 指令本身？

2.

在用户态发生时钟中断时，`scause` 的值是什么？内核应该如何处理？它与 `ecall` 产生的陷阱在处理流程上有何不同？

2.3 两个机制的协同关系

第一次从内核进入用户态，需要两个机制**串联工作**：

```
1 内核调度器（S-Mode）
2    → swtch(调度器context, proczero内核context)    ← struct context
```

```

3      → 跳入 forkret()                                ← proczero 的"出生地"
4      → forkret 调用 usertrapret()
5      → usertrapret() 配置 trapframe 并执行 sret      ← struct trapframe
6      → CPU 降权到 U-Mode, PC = proczero 用户代码地址
7
8      ↑
      这是本实验要建立的完整路径

```

第三节：S-Mode ↔ U-Mode 特权级切换的硬件机制

3.1 从 S-Mode 进入 U-Mode: `sret` 指令

`sret` 是返回用户态的唯一指令，执行时硬件自动完成：

```

1  PC          ← sepc (用户程序应该从哪里开始/继续执行)
2  特权级      ← sstatus.SPP (0=U-Mode, 1=S-Mode, 设为0即可返回U-Mode)
3  中断使能    ← sstatus.SPIE → sstatus.SIE (恢复中断使能状态)

```

因此，进入用户态前，内核必须手动设置：

1. `sepc` = 用户程序入口地址（或上次被打断的 PC）
2. `sstatus.SPP` = 0（目标是 U-Mode）
3. `sstatus.SPIE` = 1（返回后开启中断，否则用户程序永远无法被时钟中断）
4. 如果使用独立用户页表：将页表地址写入 `satp` 并刷新 TLB

3.2 从 U-Mode 返回 S-Mode: `ecall` 或各类陷阱

用户程序执行 `ecall` 时，硬件：

```

1  sepc          ← ecall 指令的 PC（注意：不是 ecall+4！）
2  scause        ← 8 (U-Mode ecall 的异常编号)
3  sstatus.SPP ← 0 (记录陷入前的特权级，U-Mode=0)
4  sstatus.SIE ← 0 (关闭中断)
5  PC           ← stvec (跳转到内核设置的陷阱处理入口)

```

关键注意事项：

- CPU 不会自动保存通用寄存器——内核的陷阱处理程序必须立即保存所有寄存器到 `trapframe`
- CPU 不会切换栈——在保存寄存器时，内核必须切换到当前进程的内核栈
- 返回用户态时，内核必须手动将 `sepc` += 4，跳过 `ecall` 指令

3.3 用户态下的陷阱入口——`stvec` 的两种工作状态

`stvec` 寄存器在本实验中需要动态切换：

当前状态	stvec 指向	原因
在内核态（S-Mode）执行	<code>sys_trap_vector</code> (Lab4 设置)	内核异常直接进入内核陷阱处理
在用户态（U-Mode）执行	用户陷阱入口地址	用户态陷阱必须先保存用户寄存器

因此 `usertrapret()` 在返回用户态之前，必须把 `stvec` 切换到用户态陷阱入口；
`usertrap()` 在进入后，必须立即把 `stvec` 切回内核陷阱入口（防止内核处理过程中发生嵌套陷阱时走错入口）。

第四节：你需要构建什么（设计蓝图）

完成整个实验后，系统的调用链应当如下图所示。动手之前，请先在纸上画出你的设计方案，标注每个函数位于哪个文件、执行在哪个特权级。

```
1  [系统启动]
2  main()
3  └─ procinit()           ← Task A：初始化进程表
4  └─ userinit()          ← Task B：创建 proczero（自主设计，新函数）
5  └─ scheduler()         ← Task A：调度器循环
6
7  [调度器首次调度 proczero]
8  scheduler()
9  └─ swtch(cpu->ctx, proczero->ctx) ← Task A
10     └─ forkret()         ← Task B：进程第一次被调度时的"出生之地"（自主设计）
11        └─ usertrapret() ← Task C：准备并执行 sret 进入 U-Mode（自主设计）
12
13  [U-Mode：proczero 用户代码运行]
14  proczero:              ← Task D：用户态程序（自主设计，新文件）
15      ecall               # 第一次系统调用
16      ecall               # 第二次系统调用
17      dead loop
18
19  [U-Mode ecall → S-Mode]
20  usertrap()              ← Task E：用户态陷阱处理（扩展框架现有函数）
21  └─ 识别 scause=8 (ecall)
22  └─ 打印系统调用信息
23  └─ 处理时钟中断（维持 Tick!）
24  └─ usertrapret()        ← 返回 U-Mode
```

第五节：任务实施

任务 A / 框架 TODO — 完成内核态调度基础

这部分沿用现有框架的 TODO 标注。

A1. kernel/proc/proc.c — procinit() 和 allocproc()

找到 TODO [Lab5-任务1-步骤1,2] 。

procinit()：遍历全局进程表 proc[]，将每个进程的 status 初始化为 TASK_FREE。这是进程表的“出厂重置”。

allocproc()：在进程表中找到一个 TASK_FREE 的槽位，完成初始化：

1. 分配进程 ID：调用 allocpid()
2. 分配 trapframe 页：调用 kalloc()；若失败则将状态恢复为 TASK_FREE 并返回 0
3. 将进程状态设为 TASK_ALLOCATED

关键问题（回答后再动手）：

- trapframe 为什么需要独立的物理页？能否放在进程的内核栈上？
- allocproc() 为什么不直接把状态设为 TASK_READY？什么时候才变成 TASK_READY？

A2. kernel/proc/swtch.S — 内核上下文切换

找到 TODO [Lab5-任务2-步骤1,2] 。

参考 proc.h 中 struct context 的字段顺序，swtch 需要按相同的内存偏移量保存和恢复 s0 - s11、ra、sp 共 14 个寄存器。框架已给出前 4 个（s0 - s3）的示例。

偏移量规律（从 struct context 定义直接推导）：

```
1 s0 =偏移0,   s1 =偏移8,   s2 =偏移16,  s3 =偏移24
2 s4 =偏移32,  s5 =偏移40,  s6 =偏移48,  s7 =偏移56
3 s8 =偏移64,  s9 =偏移72,  s10 =偏移80, s11 =偏移88
4 ra =偏移96,  sp =偏移104
```

保存（sd）和恢复（ld）的顺序必须完全镜像对称，最后执行 ret。

关键问题：

- swtch 返回后，CPU 执行的是哪个进程的代码？为什么？
- 如果 ra 寄存器保存的是 forkret 的地址，第一次 swtch 后会发生什么？

A3. kernel/proc/proc.c — scheduler() 和 yield()

找到 TODO [Lab5-任务3] 和 TODO [Lab5-任务4] 。

scheduler()：无限循环，遍历进程表，找到 TASK_READY 的进程：

1. 将其状态改为 TASK_RUNNING
2. 设置 c->proc = p
3. 调用 swtch(&c->context, &p->context) 切换过去
4. swtch 返回（进程放弃 CPU 后）：清零 c->proc

yield()（配合 Lab4 时钟中断）：将当前进程状态改为 TASK_READY，然后切回调度器：

swtch(&p->context, &mycpu()->context)

完成 Task A 后，在 Lab4 的 case 1: 中补充对 yield() 的调用：

```
1 if (myproc() != 0 && myproc()->status == TASK_RUNNING)
2     yield();
```

任务 B — 自主设计：创建第一个进程 `proczero`（无框架模板）

这是本实验最核心的设计挑战。框架中没有 `userinit()` 或 `forkret()` 的实现，你需要自行设计并在合适的文件中实现这两个函数。

B1. 思考：进程第一次被调度时需要发生什么？

`scheduler()` 通过 `swtch` 切换到某个进程时，该进程的 `context.ra` 寄存器决定了“第一次醒来跳到哪里”。

对于新创建的进程，我们希望它第一次被调度时：

1. 先执行一段内核初始化代码（称为 `forkret`）
2. `forkret` 再调用 `usertrapret()` 完成 S→U 的最后一步

因此，`allocproc()` 必须将新进程的 `context.ra` 初始化为 `forkret` 的地址，将 `context.sp` 初始化为内核栈顶。

`forkret()` 的职责（你来实现）：

- 为什么 `forkret` 存在？直接在 `allocproc()` 中设置 `context.ra = usertrapret` 不行吗？（提示：思考 `scheduler()` 的锁机制）
- `forkret` 需要做什么才能安全地调用 `usertrapret()`？

B2. 思考：如何分配内核栈？

每个进程需要一个内核栈，用于在内核态处理陷阱时使用。这个栈必须在进程创建时分配。

框架中 `struct proc` 已经有 `kstack` 字段（内核栈顶地址）。你需要：

1. 为 `proczero` 的内核栈分配物理内存（使用 `kalloc()`）
2. 将栈顶地址存入 `p->kstack`
3. 将 `context.sp` 初始化为 `p->kstack`

问题：内核栈是 `kstack ~ kstack - PGSIZE`（栈向低地址生长），`context.sp` 应初始化为哪个值？

B3. 参考：xv6 如何创建第一个进程

在设计 `userinit()` 之前，先理解 xv6 中等价函数的工作原理。xv6 也有一个

`userinit()` 函数专门用于创建系统中第一个用户态进程（`initcode`），其逻辑如下：

xv6 `userinit()` 的工作流程：

```
1 1. 调用 allocproc()
2   → 在进程表中找一个空槽，分配 pid，分配 trapframe 页，
3     将 context.ra 设为 forkret（进程第一次被调度时的跳转目标）
4
5 2. 创建用户页表
6   → 调用 uvmcreate() 分配一张全新的空白页表
7   → 调用 uvminit(pagetable, initcode_bytes, sizeof(initcode))
8     将编译好的用户代码字节直接拷贝进页表的虚拟地址 0 处
```

```

9      (建立 VA:0 → 某物理页 的映射, 权限: R/W/X/U)
10
11 3. 初始化 trapframe
12   → p->trapframe->epc = 0          # 用户程序从虚拟地址 0 开始执行
13   → p->trapframe->sp  = PGSIZE      # 用户栈顶 (用户代码占了第0页, 栈从第1页顶部向下)
14
15 4. 设置进程元信息
16   → strncpy(p->name, "initcode", ...)
17   → p->state = RUNNABLE            # 允许被调度器选中
18
19 5. 此后 scheduler() 找到它 → swtch 进入它的 context → 跳入 forkret
20   → forkret 调用 usertrapret → sret 进入 U-Mode → PC = 0

```

`struct proc` 字段对照 (来自本框架的 `proc.h`) :

字段	类型	作用	你需要在 <code>userinit()</code> 中设置
<code>status</code>	<code>enum task_status</code>	进程生命周期状态	最终设为 <code>TASK_READY</code>
<code>pid</code>	<code>int</code>	进程 ID	<code>allocproc()</code> 已设置
<code>pagetable</code>	<code>pagetable_t</code>	用户页表根地址	分配并映射用户代码
<code>trapframe</code>	<code>struct trapframe *</code>	用户寄存器快照	<code>allocproc()</code> 已分配页, 需初始化字段
<code>trapframe->epc</code>	<code>uint64</code>	用户程序计数器	= 用户代码入口地址
<code>trapframe->sp</code>	<code>uint64</code>	用户栈指针	= 用户栈顶地址
<code>context</code>	<code>struct context</code>	内核调度上下文	<code>allocproc()</code> 已初始化 <code>ra=forkret</code>
<code>kstack</code>	<code>uint64</code>	内核栈顶地址	需分配物理页并记录
<code>sz</code>	<code>uint64</code>	用户地址空间大小	= 分配给用户的内存大小 (字节)
<code>name</code>	<code>char[16]</code>	调试用进程名	设为 <code>"proczero"</code>

本框架的简化策略 (与 xv6 的最小差异) :

xv6 为每个进程创建完全独立的用户页表 (`uvmcreate`)。在本实验的简化版本中, 可以跳过这一步: 直接在内核恒等映射的地址空间中为用户代码分配一个物理页 (`kalloc()`), 并通过 `mappages()` 在内核页表中为这个物理页添加 `PTE_U` 权限位, 使 CPU 在 U-Mode 下能

访问到它。这样 `trapframe->epc` 就直接填写该物理页的地址（在恒等映射下等于虚拟地址）。

Lab 6 中将实现独立用户地址空间，届时用户程序的虚拟地址 0 才真正与内核地址空间隔离。

设计并实现 `userinit()` 函数，该函数必须完成以下工作（具体实现方式由你决定）：

1. 调用 `allocproc()` 获取已初始化基础字段的 PCB。
2. 分配 `proczero` 内核栈（`kalloc()`），将栈顶地址存入 `p->kstack`，同时更新 `p->context.sp = p->kstack`。
3. 为用户代码分配物理内存（`kalloc()`），将 `proczero_code[]` 字节数组拷贝进去（`memcpy()`），并在页表中建立映射（包含 `PTE_U` 权限）。
4. 为用户栈分配物理内存（`kalloc()`），在页表建立映射（含 `PTE_U`）。
5. 初始化 `p->trapframe`：`epc` = 用户代码入口，`sp` = 用户栈顶，其余清零。
6. 设置进程名称 `p->name = "proczero"`，并将状态设为 `TASK_READY`。

思考后再动手： 在步骤3中，为什么需要将代码拷贝到新分配的物理页，而不能直接让页表映射到 `proczero_code[]` 数组本身在内核 `.data` 段中的地址？如果直接映射，会有什么问题？

任务 C — 自主设计：`usertrapret()` 进入用户态（无框架模板）

`usertrapret()` 是从 S-Mode 跳入 U-Mode 的最后一步。它已在 `defs.h` 中声明，但尚未实现。

在 `trap.c` 中实现 `usertrapret()`，该函数必须：

1. **关闭中断：**防止在配置 `stvec` 的间隙被时钟中断打断而进入错误的处理路径。
2. **切换 `stvec` 到用户态陷阱入口：**用户态下若发生陷阱，应该跳到能保存用户寄存器的处理程序。
 - 在简化实现中，可以直接设置 `stvec` 到一个你编写的用户态陷阱汇编入口（类似 `kernelvec.S` 的结构，但保存到 `trapframe` 而非栈）
 - 或者，直接让 `stvec` 指向 `usertrap()`（更简单，但意味着不保存/恢复用户通用寄存器）
3. **填充 `trapframe` 的内核信息字段：**
 - `p->trapframe->kernel_sp` = 内核栈顶（`p->kstack`），使进入内核时有栈可用
 - `p->trapframe->kernel_trap` = `usertrap` 的地址
4. **配置 `sstatus`：**
 - 清除 `SPP` 位（bit 8）：确保 `sret` 后进入 U-Mode
 - 设置 `SPIE` 位（bit 5）：`sret` 后开启中断（否则用户态永远屏蔽中断）
 - 参考 `riscv.h` 中的 `SSTATUS_SPP` 和 `SSTATUS_SPIE` 宏
5. **设置 `sepc` = `p->trapframe->epc`**（用户程序要从哪里开始执行）
6. **执行 `sret`：**切换到 U-Mode，PC 跳到 `sepc`

关键设计问题：此时需要切换页表吗（写 `satp`）？在简化版本中，如果用户代码使用内核恒等映射地址空间，则不需要；未来 Lab 6 实现独立用户地址空间后，此处必须切换。

任务 D — 自主设计： `proczero` 用户态程序（新文件）

这是运行在 U-Mode 下的第一个用户程序。它非常简单：

行为要求：

1. 执行第一次系统调用（`ecall`）
2. 执行第二次系统调用（`ecall`）
3. 进入无限循环（`while(1)`）

实现选择（二选一）：

选择 A： 用 C 语言写 `user/proczero.c`，让 Makefile 用 `riscv64-unknown-elf-gcc` 编译成独立的 ELF，然后 `objcopy` 提取代码段嵌入内核（参考 xv6 的 `initcode` 处理方式）。

选择 B（更简单）： 直接在 `kernel/proc/proc.c` 或新文件中写一小段内联汇编或字节数组，包含执行两次 `ecall` 然后死循环的机器码，作为 `proczero_code[]` 数组，在 `userinit()` 中将其复制到分配的用户物理页。

选择 B 的机器码参考（RISC-V 汇编）：

```
1 # proczero 用户态代码（汇编层面的理解）
2 ecall          # 第一次系统调用 → 机器码 0x00000073
3 ecall          # 第二次系统调用 → 机器码 0x00000073
4 j .            # 死循环          → 机器码 0x0000006f
```

对应的字节数组：

```
1 static uint8 proczero_code[] = {
2     0x73, 0x00, 0x00, 0x00,    // ecall
3     0x73, 0x00, 0x00, 0x00,    // ecall
4     0x6f, 0x00, 0x00, 0x00,    // j .
5 };
```

任务 E — 扩展框架： `usertrap()` 用户态陷阱处理

`trap.c` 中已有 `usertrap()` 的空框架（Lab6-任务2 的 TODO 只处理了 `ecall` 的 PC 移位）。你需要在本实验中实现完整的用户态陷阱分发逻辑。

`usertrap()` 需要处理三类情况：

情况1 — `ecall`（`scause = 8`）： 系统调用

1. 立即将 `stvec` 切换回 `sys_trap_vector`（内核陷阱向量）
2. 将 `trapframe->epc += 4`（跳过 `ecall` 指令，返回 `ecall` 的下一条指令）
3. 开启中断（系统调用期间允许被打断）
4. 打印系统调用信息（如 "syscall from proczero (pid=1)"）
5. 调用 `usertrapret()` 返回用户态

情况2 — 软件中断 / 时钟信号（`scause = 0x8000000000000001`）：

1. 原有中断处理逻辑（清除 SSIP，打印 Tick!，必要时 `yield`）

2

之后调用 `usertrapret()` 返回用户态

情况3 — 其他异常:

1

打印错误信息 (`scause`, `sepc`, `stval`) , `panic` 或终止进程

设计问题（必须在实验报告中回答）：

- 为什么 `usertrap()` 进入后第一件事是把 `stvec` 切换回 `sys_trap_vector` ? 如果不做这步, 会有什么风险?
- 时钟中断为什么也可能进入 `usertrap()` ? (提示: 想想 `mideleg` 设置了哪些委托)

第六节：需要修改的地方汇总

完成本实验，你的改动应当涉及（但不限于）以下位置：

位置	内容
<code>kernel/proc/proc.c</code>	完成 <code>procinit()</code> <code>allocproc()</code> <code>scheduler()</code> <code>yield()</code> ; 新增 <code>userinit()</code> <code>forkret()</code>
<code>kernel/proc/swtch.S</code>	完成寄存器保存/恢复
<code>kernel/trap/trap.c</code>	在 <code>case 1:</code> 加 <code>yield()</code> (Lab4 遗留) ; 实现 <code>usertrap()</code> ; 新增 <code>usertrapret()</code>
<code>kernel/boot/main.c</code>	在 <code>start_main()</code> 中调用 <code>procinit()</code> 和 <code>userinit()</code> ; 用 <code>scheduler()</code> 替代 <code>while(1)</code>
<code>user/proczero.c</code> 或同等	用户态程序 (两次 <code>ecall</code> + 死循环)

Makefile 可能需要修改 以编译和链接新增的用户程序文件。

第七节：调试指南

工具准备

```
1 # 在一个终端启动 QEMU 并等待 GDB 连接 ( Makefile 通常提供此目标)
2 make qemu-gdb
3
4 # 在另一个终端启动 GDB
5 riscv64-unknown-elf-gdb kernel/kernel
6 (gdb) target remote :1234
7 (gdb) b usertrapret      # 断点在进入用户态前
8 (gdb) b usertrap        # 断点在用户态陷阱处理
9 (gdb) c
```

分阶段验证策略

阶段	验证方法
Task A 完成后	创建两个简单内核线程（类似 Lab4 的 Tick，改为两个轮转打印），确认 <code>swtch</code> 和 <code>scheduler</code> 正常工作
<code>allocproc</code> / <code>userinit</code> 完成后	在 GDB 中检查 <code>proc[0].trapframe->epc</code> 、 <code>proc[0].kstack</code> 、 <code>proc[0].context.ra</code> 的值是否符合预期
<code>usertrapret</code> 完成后	GDB 断点在 <code>sret</code> 指令，检查 <code>sepc</code> 、 <code>sstatus</code> 的值
进入 U-Mode 后	GDB <code>stepi</code> 单步执行，确认 PC 跳到了 <code>proczero</code> 代码地址； 查看 <code>csrr a0, sstatus</code> 确认 SPP=0
<code>ecall</code> 触发后	GDB 断点在 <code>usertrap</code> ，确认 <code>scause = 8</code> ， <code>sepc</code> 指向 <code>ecall</code> 指令地址

常见症状与根因

症状	最可能的根因
系统静默死机（无任何输出）	进入用户态后发生了 page fault，且 <code>stvec</code> 未正确设置导致无法返回内核处理
<code>usertrap: unexpected scause</code>	用户程序发生了意外异常（如指令地址未对齐）；检查 <code>proczero</code> 代码的地址对齐
<code>sret</code> 后系统立刻崩溃	<code>sepc</code> 地址不在有效的用户代码范围内，或该地址没有 U 权限的页表条目
时钟中断消失（无 <code>Tick!</code> ）	<code>usertrapret()</code> 没有正确设置 <code>sstatus.SPIE</code> ，导致 U-Mode 下中断被永久屏蔽
<code>ecall</code> 无输出，系统卡住	<code>usertrap()</code> 中没有实现 <code>scause=8</code> 的分支；或 <code>trapframe->epc</code> 没有 <code>+=4</code> 导致死循环 <code>ecall</code>
两次 <code>ecall</code> 但只输出一次	<code>usertrapret()</code> 返回后 PC 未正确恢复，导致第二次 <code>ecall</code> 没有被执行

第八节：思考题（实验报告必答）

- `context` vs `trapframe` 的根本区别：为什么内核线程切换只需要保存 14 个寄存器，而用户态陷阱需要保存全部 32 个？从 RISC-V ABI 和信任模型两个角度分析。
- `forkret` 的必要性：如果直接设置 `context.ra = usertrapret`，而不通过 `forkret` 中转，会有什么问题？（提示：想想 `scheduler` 在 `swtch` 前后可能持有什么资源）
- `stvec` 的双重工作状态：画出以下场景中 `stvec` 的状态变化时序图：从 `usertrapret()` 执行 `sret` 进入用户态，用户程序执行 `ecall`，到 `usertrapret()` 再次返回用户态的完整循环。为什么必须在 `usertrap()` 入口处立即切换 `stvec`？
- 用户栈与内核栈：本实验中 `proczero` 有几个栈？分别是什么，存在哪里，在什么时候使用？
- `sepc + 4` 的语义：为什么从 `ecall` 返回时需要将 `sepc` 加 4？如果不加会怎样？如果某个异常（如缺页故障）处理完后，`sepc` 应该加 4 吗？

第九节：实验完成后的系统全貌

`start_main()` 在本实验完成后的形态：

```
1 void start_main() {
2     kinit(); kvmininit(); kvminithart(); // Lab3：内存与分页
3     trapinithart();                      // Lab4：注册陷阱向量
4     intr_on();                            // Lab4：开启 S-Mode 中断
5     procinit();                          // Lab5：初始化进程表
6     userinit();                          // Lab5：创建第一个用户进程
```

```
7     scheduler();                                // Lab5 : 进入调度器 (永不返回)
8 }
```

Lab6 预告： 下个实验中，你将把当前手工嵌入的 `proczero` 机器码替换为真正的用户程序，实现完整的系统调用分发表（`syscall()` 函数），并实现 `sys_getpid()`、`sys_write()` 等具有实际功能的系统调用——操作系统的用户态编程接口从此真正建立。